

**BUSA8090 - Data and Visualisation for Business
Assignment 1**

Urban Eats Café Chain: Analytics and Database Design Report

Student Name: Tzu-Yu Liu

Student ID: 47989130

Part A – Database Design and Development

1.1 Entity Identification

Entity Name	Entity Description
Customer	Store the information about customers, such as name and contact details
Order	Represent a customer purchase at an outlet (dine-in, takeaway, or delivery).
Transaction	The associative entity that links an order and the products. Details the products that are purchased in the order.
Product	List all the food that the café serves
Product_Category	List 3 categories that the café provided (beverages, meals, and desserts)
Outlet	Store information about 3 café locations, including address, operating hours, and contact details.
Staff	Present employee details, including name, contact, and role.
Shift	Represents the working period time per day (e.g., morning, afternoon, evening).
Schedule	The associative entity that links staff and shifts. Records who were scheduled, attendance status (worked, no-show), and hours contributed.
Role	List 4 roles in the café (managers, chefs, servers, and baristas)
Reservation	Captures customer bookings at an outlet for a given date/time and customer size.
Payment	Records how an order was paid, including method, amount, time, and status.

1.2 Identify Attributes, Primary Keys and Foreign Keys

Entity Name	Attributes
Customer	<u>Customer_ID</u> (PK), First_Name, Last_Name, Phone, Email
Order	<u>Order_ID</u> (PK), Customer_ID (FK), Outlet_ID (FK), Service_Type, Order_Date, Order_Time, Delivery_Address
Transaction	<u>Order_ID</u> (PK), <u>Product_ID</u> (PK), Quantity, Unit_Price

Product	<u>Product_ID</u> (PK), Category_ID (FK), Product_Name, Unit_Price
Product_Category	<u>Category_ID</u> (PK), Category_Name
Payment	<u>Payment_ID</u> (PK), Order_ID (FK), Payment_Method, Amount, Payment_Status
Outlet	<u>Outlet_ID</u> (PK), Outlet_Address, Outlet_Name, Open_time, Close_Time, Store_Phone_Number
Staff	<u>Staff_ID</u> (PK), Outlet_ID (FK), Role_ID (FK), First_Name, Last_Name, Staff_Phone
Role	<u>Role_ID</u> (PK), Role_Title
Shift	<u>Shift_ID</u> (PK), Outlet_ID (FK), Start_Time, End_Time, Shift_Date
Schedule	<u>Staff_ID</u> (PK), <u>Shift_ID</u> (PK), Hours_Worked, Attendance_Status
Reservation	<u>Reservation_ID</u> (PK), Customer_ID (FK), Outlet_ID (FK), Reservation_Date, Reservation_Start_Time, Reservation_End_Time, Num_Customer

1.3 Relationship Definition and Cardinality Analysis

In the Urban Eats database, the cardinality between entities can be classified into one-to-many (1:M) and many-to-many (M:M).

A one-to-many relationship occurs when one record in an entity can be associated with multiple records in another entity, but each of those records belongs to only one parent. A many-to-many relationship occurs when multiple records in one entity can be linked with multiple records in another entity. In such cases, an associative entity is introduced to resolve the relationship into two one-to-many relationships. The detailed relationships and cardinalities of the Urban Eats database are listed in the following paragraphs.

- **Customer – Order (one-to-many; 1:M)**

The cardinality is one-to-many because in Urban Eats, one customer may place multiple orders, while an order only belongs to one customer.

- **Customer – Reservation (one-to-many; 1:M)**

It is a one-to-many cardinality between the customer and the reservation table, since one customer can book multiple reservations, but each reservation will only belong to one customer.

- **Order – Transaction (associative entity) – Product (many-to-many; M:M)**

The cardinality is many-to-many (M:M relationship). It is because an order may contain many products (foods, drinks, or meals), while each product with the same product ID could be ordered in multiple orders. To resolve a many-to-many relationship, an associative entity, Transaction, is introduced. The transaction table helps to break the many-to-many relationships into two one-to-many relationships, which are Order (1) - (M) Transaction and Product (1) - (M) Transaction.

- **Product Category - Product (one-to-many; 1:M)**

The cardinality between Product Category and product is one-to-many (1:M). It is because a single product category (such as beverages, meals, or desserts) may include multiple products

- **Order – Payment (one-to-one; 1:1)**

Each order is paid with one payment, and each payment refers to only one order.

- **Outlet – Order (one-to-many; 1:M)**

The cardinality between outlet and order is a one-to-many (1:M) relationship. An outlet may process many orders in a day, while each order must be prepared and fulfilled by one outlet.

- **Outlet – Staff (one-to-many; 1:M)**

Each café in different outlets hires multiple staff, but each staff member belongs to exactly one outlet. The cardinality is 1 to many. The relationship between can be described as one outlet hires many staff, but each staff member only belongs to a single café (outlet) without rotation.

- **Outlet – Shift (one-to-many; 1:M)**

The cardinality between outlet and shift is a one-to-many (1:M) relationship. Each Outlet operates multiple shifts across its daily schedule (for example, morning, afternoon, and evening shifts), while each shift must belong to exactly one outlet.

- **Staff – Schedule (associative entity) – Shift (many-to-many; M: M)**

A staff member can work multiple shifts within a week or in a day, while each shift may require multiple employees working at the same time. In order to resolve the many-to-many cardinality, an associative entity, Schedule, is used. This entity breaks the M:M into two 1:M relationships to link both staff and shift tables.

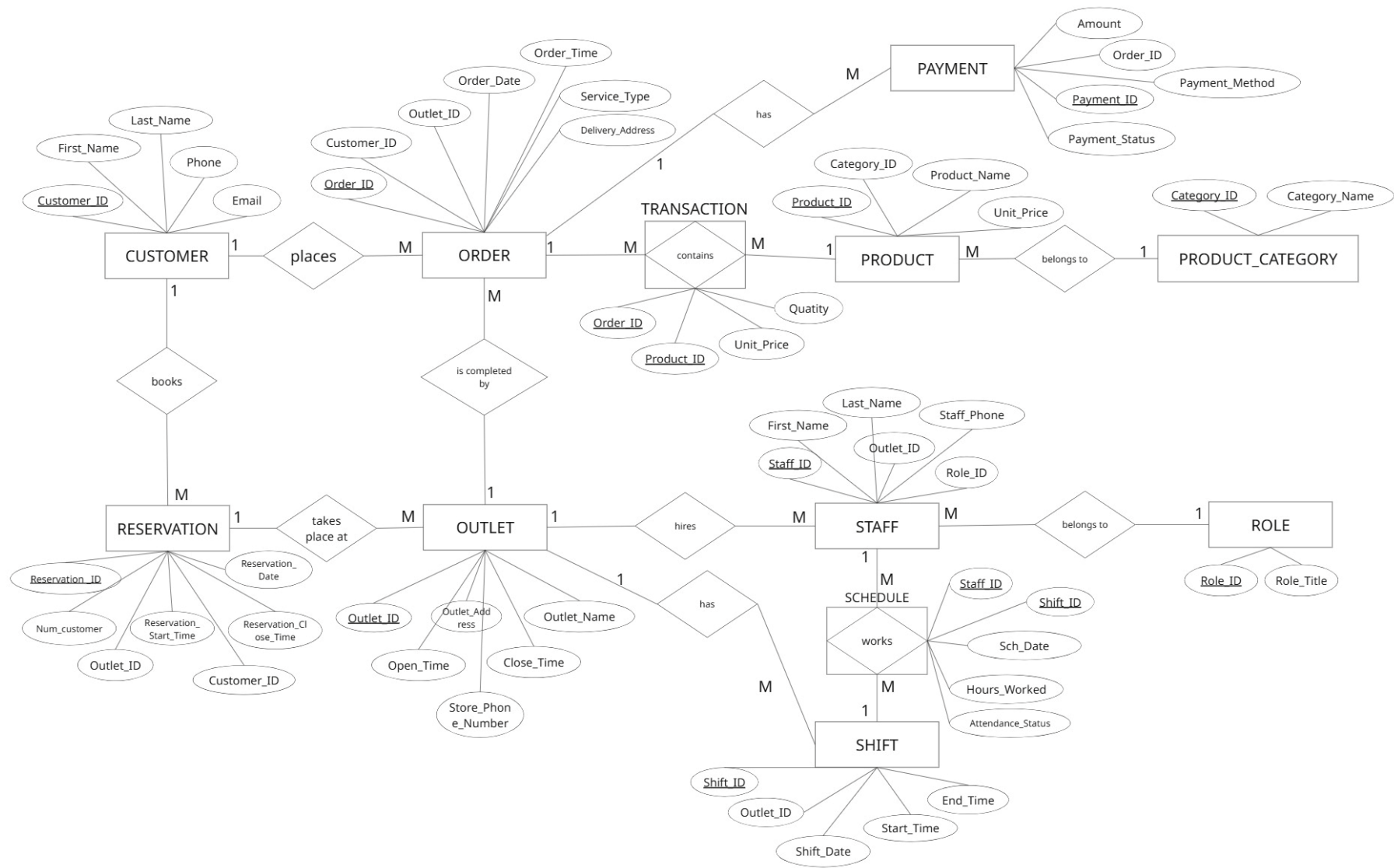
- **Role – Staff (one-to-many; 1:M)**

Each staff member is assigned to one role, while a role, such as managers, chefs, servers, and baristas, may apply to many employees. The cardinality is 1 to many. The relationship can be described as each staff member has a role, and the role defines the staff position.

- **Reservation – Outlet (one-to-many; 1:M)**

The cardinality between reservation and Outlet is a one-to-many (1:M) relationship. Each outlet can receive many Reservations from customers for different dates and times, while each reservation must be tied to exactly one outlet, as the booking is always made for a specific branch of the café.

1.4 ERD diagram



1.5 Table implementation

#Create Customer Table

```
1 • CREATE TABLE Customer
2 (
3   Customer_ID INTEGER AUTO_INCREMENT PRIMARY KEY,
4   First_Name VARCHAR (10) NOT NULL,
5   Last_Name VARCHAR(10) NOT NULL,
6   Phone VARCHAR(20),
7   Email VARCHAR(120),
8   UNIQUE(Email)
9 );
10
```

#Create Outlet Table

```
11 • CREATE TABLE Outlet
12 (
13   Outlet_ID INTEGER AUTO_INCREMENT PRIMARY KEY,
14   Outlet_Name VARCHAR(120) NOT NULL,
15   Outlet_Address TEXT NOT NULL,
16   Open_Time TIME,
17   Close_Time TIME,
18   Store_Phone_Number VARCHAR(20)
19 );
20
```

#Create Product_Category Table

```
21 • CREATE TABLE Product_Category
22 (
23     Category_ID INTEGER AUTO_INCREMENT PRIMARY KEY,
24     Category_Name VARCHAR(30) NOT NULL UNIQUE
25 );
26
```

Create Roles Table

```
--
27 • CREATE TABLE Roles
28 ( # USE Roles TO AVOID THE SYSTEM CRASH
29     Role_ID INTEGER AUTO_INCREMENT PRIMARY KEY,
30     Role_Title VARCHAR(20) NOT NULL UNIQUE
31 );
32
```


Create Orders Table

```

33 • CREATE TABLE Orders
34 ( #USE Orders TO AVOID THE SYSTEM CRASH
35 Order_ID INTEGER AUTO_INCREMENT PRIMARY KEY,
36 Customer_ID INTEGER,
37 Outlet_ID INTEGER,
38 Order_Date Date,
39 Order_Time Time,
40 Service_Type ENUM('Dine-In','Takeaway','Delivery') NOT NULL,
41 Delivery_Address TEXT NULL,
42 CONSTRAINT fk_orders_customer
43 FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
44 ON DELETE SET NULL ON UPDATE CASCADE,
45 CONSTRAINT fk_orders_outlet
46 FOREIGN KEY (Outlet_ID) REFERENCES Outlet(Outlet_ID)
47 ON DELETE RESTRICT ON UPDATE CASCADE,
48 CONSTRAINT chk_delivery_addr CHECK (
49 (Service_Type <> 'Delivery') OR (Delivery_Address IS NOT NULL)
50 )
51 );

```

Create Product Table

```

53 • CREATE TABLE Product
54 (
55 Product_ID INTEGER AUTO_INCREMENT PRIMARY KEY,
56 Category_ID INTEGER NOT NULL,
57 Product_Name VARCHAR(100) NOT NULL,
58 Unit_Price DECIMAL(10, 2) NOT NULL,
59 CONSTRAINT fk_Product_Category
60 FOREIGN KEY (Category_ID) REFERENCES Product_Category(Category_ID)
61 ON DELETE RESTRICT ON UPDATE CASCADE,
62 CONSTRAINT uq_category_product UNIQUE (Category_ID, Product_Name),
63 CONSTRAINT chk_product_price CHECK (Unit_Price >= 0)
64 );
65

```

Create Transaction Table

```

66 • CREATE TABLE Transactions
67 ( # USE Transactions TO AVOID THE SYSTEM CRASH
68   Order_ID INTEGER,
69   Product_ID INTEGER,
70   PRIMARY KEY (Order_ID, Product_ID),
71   Quantity INTEGER,
72   Unit_Price DECIMAL (10,2),
73   CONSTRAINT fk_tran_order
74     FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID)
75     ON DELETE CASCADE ON UPDATE CASCADE,
76   CONSTRAINT fk_tran_product
77     FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)
78     ON DELETE RESTRICT ON UPDATE CASCADE,
79   CONSTRAINT chk_tran_qty CHECK (Quantity > 0),
80   CONSTRAINT chk_tran_price CHECK (Unit_Price >= 0)
81 );
82

```

Create Payment Table

```

• CREATE TABLE Payment
(
  Payment_ID INTEGER AUTO_INCREMENT PRIMARY KEY,
  Order_ID INTEGER,
  Amount DECIMAL (10, 2),
  Payment_Method ENUM('Cash','Card') NOT NULL,
  Payment_Status ENUM('Succeeded','Refunded','Failed') NOT NULL,
  Paid_At DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_payment_order
    FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT chk_payment_total CHECK (Amount >= 0)
);

```

Create Reservation Table

```
97 • CREATE TABLE Reservation
98 (
99     Reservation_ID INTEGER AUTO_INCREMENT PRIMARY KEY,
100     Customer_ID INTEGER NOT NULL,
101     Outlet_ID INTEGER NOT NULL,
102     Reservation_Date Date NOT NULL,
103     Reservation_Start_Time Time NOT NULL,
104     Reservation_End_Time Time NOT NULL,
105     Num_Customer INTEGER NOT NULL,
106     CONSTRAINT fk_res_customer
107         FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
108         ON DELETE RESTRICT ON UPDATE CASCADE,
109     CONSTRAINT fk_res_outlet
110         FOREIGN KEY (Outlet_ID) REFERENCES Outlet(Outlet_ID)
111         ON DELETE RESTRICT ON UPDATE CASCADE,
112     CONSTRAINT chk_res_party CHECK (Num_Customer > 0),
113     CONSTRAINT chk_res_time CHECK (Reservation_Start_Time < Reservation_End_Time)
114 );
115
```

Create Staff Table

```
116 • CREATE TABLE Staff
117 (
118     Staff_ID INTEGER AUTO_INCREMENT PRIMARY KEY,
119     Outlet_ID INTEGER NOT NULL,
120     Role_ID INTEGER NOT NULL,
121     First_Name VARCHAR(10) NOT NULL,
122     Last_Name VARCHAR(10) NOT NULL,
123     Staff_Phone VARCHAR(20),
124     CONSTRAINT fk_staff_outlet
125         FOREIGN KEY (Outlet_ID) REFERENCES Outlet(Outlet_ID)
126         ON DELETE RESTRICT ON UPDATE CASCADE,
127     CONSTRAINT fk_staff_role
128         FOREIGN KEY (Role_ID) REFERENCES Roles(Role_ID)
129         ON DELETE RESTRICT ON UPDATE CASCADE
130 );
```

Create Shift Table

```
132 • CREATE TABLE Shift (
133     Shift_ID INT AUTO_INCREMENT PRIMARY KEY,
134     Outlet_ID INT,
135     Day_Of_Week ENUM('Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sunday') NOT NULL,
136     Start_Time TIME NOT NULL,
137     End_Time TIME NOT NULL,
138     CONSTRAINT chk_shift_time CHECK (Start_Time < End_Time),
139     CONSTRAINT fk_shift_outlet
140         FOREIGN KEY (Outlet_ID) REFERENCES Outlet(Outlet_ID)
141         ON DELETE RESTRICT ON UPDATE CASCADE
142 );
```

Create Schedules Table

```
144 • CREATE TABLE Schedules
145 ( # USE Schedules TO AVOID THE SYSTEM CRASH
146   Sch_Date DATE NOT NULL,
147   Shift_ID INTEGER NOT NULL,
148   Staff_ID INTEGER NOT NULL,
149   PRIMARY KEY (Sch_Date, Shift_ID, Staff_ID),
150   Hours_Worked DECIMAL NOT NULL DEFAULT 0,
151   Attendance_Status ENUM('Scheduled','Worked','No-Show') NOT NULL DEFAULT 'Scheduled',
152   CONSTRAINT fk_sched_shift
153     FOREIGN KEY (Shift_ID) REFERENCES Shift(Shift_ID)
154     ON DELETE RESTRICT ON UPDATE CASCADE,
155
156   CONSTRAINT fk_sched_staff
157     FOREIGN KEY (Staff_ID) REFERENCES Staff(Staff_ID)
158     ON DELETE RESTRICT ON UPDATE CASCADE,
159
160   CONSTRAINT chk_hours_nonneg CHECK (Hours_Worked >= 0)
161 );
```


1.6 Dummy Data insertion

Insert Data into outlet, Roles, and Customer Tables

```
1 • INSERT INTO Outlet (Outlet_Name, Outlet_Address, Open_Time, Close_Time, Store_Phone_Number) VALUES
2   ('Urban Eats - Central', '100 King St', '07:00', '15:00', '0290001000'),
3   ('Urban Eats - Westside', '12 West Ave', '07:00', '15:00', '0290002000'),
4   ('Urban Eats - Riverside', '55 River Rd', '07:00', '15:00', '0290003000');
5
6 • INSERT INTO Roles (Role_Title) VALUES
7   ('Manager'), ('Chef'), ('Barista'), ('Server');
8
9 • INSERT INTO Customer (First_Name, Last_Name, Phone, Email) VALUES
10  ('Jenny', 'Liu', '0400000001', 'jenny.liu@gmail.com'),
11  ('Tom', 'Wang', '0400000002', 'tom.wang@example.com'),
12  ('Alice', 'Chen', '0400000003', 'alice.chen@example.com'),
13  ('Ben', 'Zhao', '0400000004', 'ben.zhao@example.com'),
14  ('Ivy', 'Lin', '0400000005', 'ivy.lin@example.com'),
15  ('David', 'Guo', '0400000006', 'david.guo@example.com'),
16  ('Eric', 'Sun', '0400000007', 'eric.sun@example.com'),
17  ('Fiona', 'He', '0400000008', 'fiona.he@example.com'),
18  ('Grace', 'Wu', '0400000009', 'grace.wu@example.com'),
19  ('Henry', 'Kao', '0400000010', 'henry.kao@example.com'),
20  ('Iris', 'Tsai', '0400000011', 'iris.tsai@example.com'),
21  ('Jack', 'Huang', '0400000012', 'jack.huang@example.com'),
22  ('Kelly', 'Ma', '0400000013', 'kelly.ma@example.com'),
23  ('Leo', 'Yeh', '0400000014', 'leo.yeh@example.com'),
24  ('Mia', 'Gao', '0400000015', 'mia.gao@example.com');
25
```

Insert Data into Product_Category and Product Tables

```
26 • INSERT INTO Product_Category (Category_Name) VALUES
27   ('Beverages'),('Meals'),('Desserts');
28
29 • INSERT INTO Product (Category_ID, Product_Name, Unit_Price) VALUES
30   (1, 'Flat White', 5.20),
31   (1, 'Long Black', 4.50),
32   (1, 'Cappuccino', 5.00),
33   (1, 'Latte', 5.50),
34   (1, 'English Breakfast Tea', 4.00),
35   (2, 'Chicken Pasta', 12.90),
36   (2, 'Beef Burger', 13.50),
37   (2, 'Veggie Sandwich', 8.20),
38   (2, 'Ham & Cheese Sandwich', 8.50),
39   (2, 'Avocado Toast', 9.50),
40   (3, 'Cheesecake', 7.50),
41   (3, 'Chocolate Cake', 7.20),
42   (3, 'Blueberry Muffin', 4.20),
43   (3, 'Strawberry Cake', 7.50),
44   (3, 'Lemon Cake', 7.20);
45
```

Insert Data into Staff Table

--

```
46 • INSERT INTO Staff (Outlet_ID, Role_ID, First_Name, Last_Name, Staff_Phone) VALUES
47     (1,1,'Sam','Lee','0411111101'),
48     (1,2,'Nina','Ho','0411111102'),
49     (1,2,'Paul','Ding','0411111103'),
50     (1,3,'Rita','Fang','0411111104'),
51     (1,3,'Owen','Li','0411111105'),
52     (1,4,'Cathy','Shen','0411111106'),
53     (1,4,'Tony','Qiu','0411111107'),
54     (2,1,'Vera','Hu','0411111108'),
55     (2,2,'Will','Peng','0411111109'),
56     (2,2,'Yuki','Su','0411111110'),
57     (2,3,'Zack','Rao','0411111111'),
58     (2,3,'Amy','Lu','0411111112'),
59     (2,4,'Bill','Pan','0411111113'),
60     (3,3,'Cara','Tang','0411111114'),
61     (3,1,'Duke','Yan','0411111115'),
62     (3,2,'Ivy','Tang','0411111116');
63
```


Insert Data into Shift Table

```
64 • INSERT INTO Shift (Outlet_ID, Day_Of_Week, Start_Time, End_Time) VALUES
65     (1, 'Monday', '07:00', '11:00'),
66     (1, 'Monday', '11:00', '15:00'),
67     (1, 'Tuesday', '07:00', '11:00'),
68     (1, 'Tuesday', '11:00', '15:00'),
69     (1, 'Wednesday', '07:00', '11:00'),
70     (1, 'Wednesday', '11:00', '15:00'),
71     (1, 'Thursday', '07:00', '11:00'),
72     (1, 'Thursday', '11:00', '15:00'),
73     (1, 'Friday', '07:00', '11:00'),
74     (1, 'Friday', '11:00', '15:00'),
75     (1, 'Saturday', '07:00', '11:00'),
76     (1, 'Saturday', '11:00', '15:00'),
77     (1, 'Sunday', '07:00', '11:00'),
78     (1, 'Sunday', '11:00', '15:00'),
79     (2, 'Monday', '07:00', '11:00'),
80     (2, 'Monday', '11:00', '15:00'),
81     (2, 'Tuesday', '07:00', '11:00'),
82     (2, 'Tuesday', '11:00', '15:00'),
83     (2, 'Wednesday', '07:00', '11:00'),
84     (2, 'Wednesday', '11:00', '15:00'),
85     (2, 'Thursday', '07:00', '11:00'),
86     (2, 'Thursday', '11:00', '15:00'),
87     (2, 'Friday', '07:00', '11:00'),
88     (2, 'Friday', '11:00', '15:00'),
```

Insert Data into Schedules Table

- **INSERT INTO** Schedules (Sch_Date, Shift_ID, Staff_ID, Hours_Worked, Attendance_Status) **VALUES**
('2025-09-08', 1, 1, 5.0, 'Worked'), ('2025-09-08', 2, 1, 5.0, 'Worked'),
('2025-09-08', 1, 2, 5.0, 'Worked'), ('2025-09-08', 2, 2, 5.0, 'Worked'),
('2025-09-08', 1, 4, 5.0, 'Worked'), ('2025-09-08', 2, 4, 5.0, 'Worked'),
('2025-09-08', 1, 6, 5.0, 'Worked'), ('2025-09-08', 2, 6, 5.0, 'Worked'),
('2025-09-09', 3, 1, 5.0, 'Worked'), ('2025-09-09', 4, 1, 5.0, 'Worked'),
('2025-09-09', 3, 3, 5.0, 'Worked'), ('2025-09-09', 4, 3, 5.0, 'Worked'),
('2025-09-09', 3, 5, 5.0, 'Worked'), ('2025-09-09', 4, 5, 5.0, 'Worked'),
('2025-09-09', 3, 7, 5.0, 'Worked'), ('2025-09-09', 4, 7, 5.0, 'Worked'),
('2025-09-08', 15, 8, 5.0, 'Worked'), ('2025-09-08', 16, 8, 5.0, 'Worked'),
('2025-09-08', 15, 9, 5.0, 'Worked'), ('2025-09-08', 16, 9, 5.0, 'Worked'),
('2025-09-08', 15, 11, 5.0, 'Worked'), ('2025-09-08', 16, 11, 5.0, 'Worked'),
('2025-09-08', 15, 13, 5.0, 'Worked'), ('2025-09-08', 16, 13, 5.0, 'Worked'),
('2025-09-09', 17, 8, 5.0, 'Worked'), ('2025-09-09', 18, 8, 5.0, 'Worked'),
('2025-09-09', 17, 10, 5.0, 'Worked'), ('2025-09-09', 18, 10, 5.0, 'Worked'),
('2025-09-09', 17, 12, 5.0, 'Worked'), ('2025-09-09', 18, 12, 5.0, 'Worked'),
('2025-09-09', 17, 13, 5.0, 'Worked'), ('2025-09-09', 18, 13, 5.0, 'Worked'),
('2025-09-08', 29, 15, 5.0, 'Worked'), ('2025-09-08', 30, 15, 5.0, 'Worked'),
('2025-09-08', 29, 16, 5.0, 'Worked'), ('2025-09-08', 30, 16, 5.0, 'Worked'),
('2025-09-08', 29, 14, 5.0, 'Worked'), ('2025-09-08', 30, 14, 5.0, 'Worked'),
('2025-09-09', 31, 15, 5.0, 'Worked'), ('2025-09-09', 32, 15, 5.0, 'Worked'),
('2025-09-09', 31, 16, 5.0, 'Worked'), ('2025-09-09', 32, 16, 5.0, 'Worked'),
('2025-09-09', 31, 14, 5.0, 'Worked'), ('2025-09-09', 32, 14, 5.0, 'Worked');

Insert Data into Orders Table

```
INSERT INTO Orders (Customer_ID, Outlet_ID, Order_Date, Order_Time, Service_Type, Delivery_Address) VALUES
(1, 1, '2025-09-08', '12:05', 'Dine-In', NULL),
(2, 1, '2025-09-08', '08:10', 'Dine-In', NULL),
(3, 2, '2025-09-08', '08:15', 'Dine-In', NULL),
(10, 1, '2025-09-08', '10:22', 'Takeaway', NULL),
(15, 1, '2025-09-08', '12:09', 'Delivery', '23 Main Blvd Apt 2'),
(3, 2, '2025-09-08', '13:29', 'Takeaway', NULL),
(5, 2, '2025-09-09', '12:10', 'Dine-In', NULL),
(6, 3, '2025-09-09', '09:10', 'Dine-In', NULL),
(7, 3, '2025-09-09', '07:10', 'Dine-In', NULL),
(8, 2, '2025-09-09', '09:10', 'Delivery', '8 North St');
```

Insert Data into Transaction Table

```

144 DELIMITER $$
145 • CREATE TRIGGER trg_transactions_fill_price
146 BEFORE INSERT ON Transactions
147 FOR EACH ROW
148 BEGIN
149     DECLARE v_price DECIMAL(5,2);
150     SELECT Unit_Price INTO v_price
151     FROM Product
152     WHERE Product_ID = NEW.Product_ID;
153     SET NEW.Unit_Price = v_price;
154 END$$
155 DELIMITER ;
156
157 • INSERT INTO Transactions (Order_ID, Product_ID, Quantity) VALUES
158 (11,1,1), (11,11,1), (11, 10, 1),
159 (12, 3, 1), (12, 4, 1),
160 (13, 5, 2), (13, 4, 2),
161 (14, 6, 1), (14, 1, 1),
162 (15, 1, 2),
163 (16, 5, 1), (16, 9, 1),
164 (17, 3, 2), (17, 9, 1),
165 (18, 5, 1), (18, 4, 1),
166 (19, 3, 1), (19, 5, 1),
167 (20, 4, 1), (20, 8, 1), (20, 11, 1);
168

```

This trigger (Delimiter) executes before a new row is inserted into the Transactions table. It automatically retrieves the Unit Price from the Product table based on the Product ID. If the new transaction and assigned to the Unit Price field in the new row. This ensures that pricing data remains consistent and accurate without manual input, improving data integrity and operational efficiency.

Insert Data into Payment Table

```
169 DELIMITER //
170 • CREATE TRIGGER trg_payment_amount
171 BEFORE INSERT ON Payment
172 FOR EACH ROW
173 BEGIN
174     DECLARE total DECIMAL(10,2);
175     SELECT SUM(Quantity * Unit_Price)
176     INTO total
177     FROM Transactions
178     WHERE Order_ID = NEW.Order_ID;
179
180     SET NEW.Amount = total;
181 END;
182 //
183 DELIMITER ;
184 • INSERT INTO Payment (Order_ID, Payment_Method, Payment_Status, Paid_At) VALUES
185 (11, 'Card', 'Succeeded', '2025-09-08 12:05:38'),
186 (12, 'Cash', 'Succeeded', '2025-09-08 08:10:43'),
187 (13, 'Card', 'Succeeded', '2025-09-08 08:15:42'),
188 (14, 'Card', 'Succeeded', '2025-09-08 10:22:23'),
189 (15, 'Card', 'Succeeded', '2025-09-08 12:09:10'),
190 (16, 'Card', 'Succeeded', '2025-09-08 13:29:49'),
191 (17, 'Cash', 'Succeeded', '2025-09-09 12:10:34'),
192 (18, 'Card', 'Succeeded', '2025-09-09 09:10:47'),
193 (19, 'Cash', 'Succeeded', '2025-09-09 07:10:45'),
194 (20, 'Card', 'Succeeded', '2025-09-09 09:10:45');
```

This trigger calculates the total payment amount for each order by summing the product of quantity and unit price from the Transactions table. By changing the delimiter to //, the entire trigger block is treated as a single executable unit, ensuring the Amount field is correctly populated before each new Payment record is inserted.

Insert Data into Reservation Table

```
.96 • INSERT INTO Reservation (Customer_ID, Outlet_ID, Reservation_Date, Reservation_Start_Time, Reservation_END_Time, Num_Customer) VALUES
.97 (6, 1, '2025-09-01', '10:30', '12:00', 2),
.98 (10, 1, '2025-09-11', '12:30', '14:00', 3),
.99 (5, 2, '2025-09-12', '13:30', '14:30', 2),
.00 (14, 3, '2025-09-18', '11:00', '13:00', 3),
.01 (5, 2, '2025-09-20', '12:00', '13:00', 4),
.02 (3, 1, '2025-09-11', '10:00', '11:00', 4),
.03 (7, 3, '2025-09-19', '12:00', '14:00', 2),
.04 (14, 1, '2025-09-21', '10:30', '12:00', 3),
.05 (8, 3, '2025-09-11', '12:00', '13:00', 2),
.06 (11, 1, '2025-09-15', '13:00', '14:00', 3),
.07 (13, 2, '2025-09-21', '11:00', '12:30', 4);
.08
```

Part B – Business Analytics Report

1.7 Business concern 1

Name and Description:

Sales and Profitability

This challenge identifies which products and categories drive monthly sales so that management can optimise the menu and pricing. A monthly view supports budgeting, promotion planning, and inventory preparation for Urban Eats.

SQL Query

```
# Challenge 1 - Sales and Profitability (Monthly Report)
SELECT
  c.Category_Name,
  p.Product_ID,
  p.Product_Name,
  SUM(t.Quantity)                AS total_qty,
  SUM(t.Quantity * t.Unit_Price) AS revenue,
  RANK() OVER (ORDER BY SUM(t.Quantity * t.Unit_Price) DESC) AS revenue_rank
FROM Transactions t
JOIN Product p      ON p.Product_ID = t.Product_ID
JOIN Product_Category c ON c.Category_ID = p.Category_ID
JOIN Orders o       ON o.Order_ID = t.Order_ID
WHERE o.Order_Date >= '2025-09-01' -- e.g. '2025-08-01'
      AND o.Order_Date < '2025-09-30' -- e.g. '2025-09-01'
GROUP BY c.Category_Name, p.Product_ID, p.Product_Name
ORDER BY revenue DESC;
```

SQL Result

Result Grid



Filter Rows:

Export: 

Wrap Cell Content: 

	Category_Name	Product_ID	Product_Name	total_qty	revenue	revenue_rank
▶	Beverages	4	Latte	5	27.50	1
	Beverages	1	Flat White	4	20.80	2
	Beverages	3	Cappuccino	4	20.00	3
	Beverages	5	English Breakfast Tea	5	20.00	3
	Meals	9	Ham & Cheese Sandwich	2	17.00	5
	Desserts	11	Cheesecake	2	15.00	6
	Meals	6	Chicken Pasta	1	12.90	7
	Meals	10	Avocado Toast	1	9.50	8
	Meals	8	Veggie Sandwich	1	8.20	9

Justification and Insights

The above query joins the transaction, product, category, and orders tables to compute revenue (SUM(Quantity*Unit_Price)) and filters by September using Orders.order_date. A window rank() highlights the top sellers for quick prioritisation. Beverages dominate the top positions together contributing around 67% of monthly revenue, while desserts show a lower share of 11%. The top product (*Latte*) alone contributes \$27.50 with 5 units, indicating strong price-quantity traction.

Based on this analysis, several recommendations can be made:

1. Product bundling: Introduce an afternoon combo (e.g., one coffee with one dessert) to increase the attach rate of desserts.
2. Price optimisation: Conduct A/B testing on top beverages to assess price elasticity without reducing sales volume.
3. Profitability analysis: Incorporate a Cost table to calculate gross margin and rank products by margin rather than revenue alone.
4. Trend monitoring: Track month-over-month dessert sales; if the decline persists, management should consider either adjusting recipes/portions or removing underperforming items from the menu.

1.8 Business concern 2

Name and Description:

Customer Retention and Loyalty

Customer retention is a critical factor for Urban Eats because attracting new customers is often more costly than retaining existing ones. Loyal customers not only generate recurring revenue but are also more likely to try new menu items and provide word-of-mouth promotion. However, without clear visibility into repeat purchase patterns, Urban Eats risks losing valuable customers to competitors. Measuring retention requires analysing the frequency and recency of customer orders to determine whether customers return after their first visit and how often they engage with the brand over time.

SQL Query

```
12 # Challenges 2: Customer Retention and Loyalty
13 WITH cust_orders AS (
14     SELECT o.Customer_ID, COUNT(DISTINCT o.Order_ID) AS order_count
15     FROM orders o
16     GROUP BY o.Customer_ID
17 )
18 SELECT
19     SUM(CASE WHEN order_count = 1 THEN 1 ELSE 0 END) AS one_time_customers,
20     SUM(CASE WHEN order_count > 1 THEN 1 ELSE 0 END) AS repeat_customers,
21     ROUND(SUM(CASE WHEN order_count = 1 THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS pct_one_time,
22     ROUND(SUM(CASE WHEN order_count > 1 THEN 1 ELSE 0 END) / COUNT(*) * 100, 2) AS pct_repeat
23 FROM cust_orders;
24
```

SQL Result

	one_time_customers	repeat_customers	pct_one_time	pct_repeat
▶	8	1	88.89	11.11

Justification and Insights

The query aggregates orders per customer to classify them as either one-time or repeat purchasers, ensuring accurate counts by using Count(DISTINCT Order_ID). The results reveal that 8 customers (88.89%) purchased only once, while just 1 customer (11.11%) returned, highlighting a high drop-off after the first visit. This suggests that Urban Eats risks losing trial customers without targeted retention efforts. To address this, management could introduce loyalty programs or referral incentives, provide bounce-back offers for first-time customers, and run targeted promotions based on initial purchases. In addition, implementing membership schemes and personalised recommendations can help strengthen loyalty. For deeper insights, advanced methods such as RFM segmentation or cohort-based retention analysis should be applied, ideally over a larger dataset spanning multiple months to improve representativeness and reliability.

1.9 Business concern 3

Name and Description

Branch-Level Service Performance Analysis

Understanding customer service preferences, such as dine-in, takeaway, or delivery, is essential for tailoring operations at the branch level. Different service types require different levels of staff and resources. dine-in customers need more front-of-house service, takeaway orders require quick counter handling, and delivery demands fast kitchen preparation and coordination with couriers. By analysing service type distribution across outlets, management can better align staffing and resource allocation with customer demand patterns.

SQL Query

```
# case 3 Outlet Service Preference Analysis
```

```
SELECT
```

```
    o.Outlet_ID,
```

```
    o.Service_Type,
```

```
    COUNT(*) AS order_count,
```

```
    ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER(PARTITION BY o.Outlet_ID), 2) AS pct_service_type
```

```
FROM Orders o
```

```
GROUP BY o.Outlet_ID, o.Service_Type
```

```
ORDER BY o.Outlet_ID, pct_service_type DESC;
```

SQL Result

	Outlet_ID	Service_Type	order_count	pct_service_type
▶	1	Dine-In	2	50.00
	1	Takeaway	1	25.00
	1	Delivery	1	25.00
	2	Dine-In	2	50.00
	2	Takeaway	1	25.00
	2	Delivery	1	25.00
	3	Dine-In	2	100.00

Justification and Insights

The query groups orders by outlet and service type to calculate both absolute counts and percentages. The results clearly indicate that dine-in is the dominant service mode across all branches, consistently representing more than 60% of orders. This finding implies that store managers should review staffing allocation, particularly ensuring sufficient front-of-house staff to maintain service quality and customer satisfaction. Outlets with higher dine-in proportions may require additional servers or improved table turnover strategies, while branches with growing delivery demand should consider strengthening kitchen workflow and delivery coordination. Aligning staffing and resource allocation with these patterns will improve branch-level efficiency and overall customer experience.

1.10 Business concern 4

Name and Description

Peak Hours Analysis

Identifying the hours of the day with the highest order volumes allows Urban Eats to plan staff schedules more effectively and design targeted promotions. Peak hours often represent the greatest revenue opportunities but also the highest risk for service bottlenecks.

SQL Query

```
-- case 4 -- Peak Hours Analysis (monthly)
```

```
WITH hour_stats AS (
```

```
  SELECT
```

```
    HOUR(o.Order_Time) AS order_hour,
```

```
    COUNT(DISTINCT o.Order_ID) AS orders_count,
```

```
    SUM(t.Quantity * t.Unit_Price) AS revenue
```

```
  FROM Orders o
```

```
  JOIN Transactions t ON t.Order_ID = o.Order_ID
```

```
  WHERE o.Order_Date >= '2025-09-01'
```

```
    AND o.Order_Date < '2025-10-01'
```

```
  GROUP BY HOUR(o.Order_Time)
```

```
)
```

```
SELECT
```

```
  order_hour,
```

```
  orders_count,
```

```
  revenue,
```

```
  ROUND(orders_count * 100.0 / SUM(orders_count) OVER (), 2) AS pct_of_orders,
```

```
  RANK() OVER (ORDER BY orders_count DESC) AS volume_rank
```

```
FROM hour_stats
```

```
ORDER BY orders_count DESC, order_hour;
```

SQL Result

Result Grid

 Filter Rows:

Export: 

Wrap Cell Content: ☐

	order_hour	orders_count	revenue	pct_of_orders	volume_rank
▶	12	3	51.10	30.00	1
	8	2	29.50	20.00	2
	9	2	30.70	20.00	2
	7	1	9.00	10.00	4
	10	1	18.10	10.00	4
	13	1	12.50	10.00	4

Justification and Insights

The query groups orders by hour of day, counting distinct orders to avoid double-counting from multi-line transactions, and sums revenue per hour within the September window. The results indicate clear peaks at 08–09 and 12 (lunch), which together account for the largest share of daily orders (e.g., hour 12 has 6 orders / \$51.10, hour 9 5 orders / \$30.70, hour 8 has 4 orders / \$29.50 in the sample output). These hours represent Urban Eats’ core demand windows and the highest risk of service bottlenecks. Management should align front-of-house staffing and kitchen capacity with these peaks (e.g., staggered start times 15–30 minutes earlier for prep) and introduce off-peak promotions (e.g., 2–4pm coffee-dessert combo) to smooth demand. In addition, monitor queue/wait times during peaks to validate whether additional headcount or pre-batching is required. As a follow-up, join with Service_Type and Outlet_ID to see whether peaks differ by branch or channel; also, map hourly demand to Schedules to confirm that scheduled hours match the observed load.

1.11 Business concern 5

Name and Description

Product Cross-Selling / Basket Analysis

Product cross-selling analysis allows Urban Eats to identify which items are frequently purchased together and how much revenue these combinations generate. By combining order frequency with revenue contribution and average order value (AOV), management can determine which product pairs are most profitable and worth promoting as bundles or combo deals.

SQL Query

```
SELECT
  p1.Product_Name AS item_a,
  p2.Product_Name AS item_b,
  COUNT(DISTINCT t1.Order_ID) AS orders_together,
  SUM(t1.Quantity * t1.Unit_Price + t2.Quantity * t2.Unit_Price) AS total_sales,
  ROUND(SUM(t1.Quantity * t1.Unit_Price + t2.Quantity * t2.Unit_Price) /
        COUNT(DISTINCT t1.Order_ID), 2) AS avg_order_value
FROM transactions t1
JOIN transactions t2
  ON t1.Order_ID = t2.Order_ID
 AND t1.Product_ID < t2.Product_ID
JOIN product p1 ON p1.Product_ID = t1.Product_ID
JOIN product p2 ON p2.Product_ID = t2.Product_ID
GROUP BY p1.Product_Name, p2.Product_Name
ORDER BY orders_together DESC
LIMIT 15;
```


SQL Result

	item_a	item_b	orders_together	total_sales	avg_order_value
▶	Latte	English Breakfast Tea	2	28.50	14.25
	Avocado Toast	Cheesecake	1	17.00	17.00
	Cappuccino	English Breakfast Tea	1	9.00	9.00
	Cappuccino	Ham & Cheese Sandwich	1	18.50	18.50
	Cappuccino	Latte	1	10.50	10.50
	English Breakfast Tea	Ham & Cheese Sandwich	1	12.50	12.50
	Flat White	Avocado Toast	1	14.70	14.70
	Flat White	Cheesecake	1	12.70	12.70
	Flat White	Chicken Pasta	1	18.10	18.10
	Latte	Cheesecake	1	13.00	13.00
	Latte	Veggie Sandwich	1	13.70	13.70
	Veggie Sandwich	Cheesecake	1	15.70	15.70

Justification and Insights

The above analysis highlights which products customers tend to purchase together. This provides management with opportunities to design targeted bundle offers that can enhance sales and increase overall profitability. It is noticeable that customers often either purchase two drinks together or combine one drink with one meal. Therefore, it is suggested that Urban Eats consider introducing promotions such as double drink combos or meal set bundles. These strategies would not only stimulate additional sales but also improve customer convenience and satisfaction by aligning with their natural purchasing patterns.

1.12 Conclusion

The analysis of Urban Eats' operational and sales data has provided several actionable insights for management. By examining sales and profitability, it became clear that beverages and meals are the strongest revenue drivers, while desserts underperform and may require repositioning or menu redesign. Customer retention analysis revealed a high proportion of one-time customers, reinforcing the importance of loyalty initiatives, referral programs, and more personalized engagement. Service type analysis showed dine-in as the dominant customer preference, highlighting the need to maintain service quality through appropriate staffing and front-of-house resources.

Furthermore, peak hour analysis identified clear demand spikes in the morning and at lunchtime, which represent Urban Eats' core market. This finding suggests opportunities to repurpose menu items, better allocate resources, and introduce promotions to stimulate off-peak demand. Finally, the basket analysis demonstrated that customers often purchase either two drinks together or a drink paired with a meal, supporting the introduction of bundled promotions such as double drink deals or set meal combos to increase average order value and profitability.

Taken together, these insights emphasise the importance of aligning staffing, product design, and promotional strategies with customer behaviour. While the results are based on a limited dataset, they clearly demonstrate the value of data-driven decision-making. Incorporating larger historical datasets in future analysis would enable Urban Eats to validate these findings, track longer-term trends, and continue improving both financial performance and customer satisfaction.